شرح الواجهة ١ — شاشة «مرحبًا»

مشروع رِزمة · Flutter · شرح سطر بسطر مع سبب كل قرار

هذا الملف يشرح كل ما كُتب لبناء أول واجهة من تصميم Figma (الإطار `مرحبًا 01`)، إضافةً إلى «الأساس» الذي تعتمد عليه كل الشاشات: الثيم، الألوان، الخطوط، الأصول، والمكوّنات المشتركة.

طريقة القراءة: كل مقطع كود يتبعه سؤالان: ماذا يفعل؟ ولماذا كُتب بهذه الطريقة؟

ملاحظة مهمة عن الاتجاه: التطبيق عربي، فنضبط اللغة على `ar` فيصبح الاتجاه RTL تلقائيًا. معنى ذلك أن أول عنصر في أي `Row` يظهر على اليمين. لذلك ترتيب العناصر في الكود هو ترتيب القراءة العربي لا ترتيب الرسم في Figma (الذي كان من اليسار لليمين).

ترتيب الملفات: الإعداد (pubspec) ← نقطة الدخول ← التطبيق ← الثيم والألوان والخطوط ← الأصول ← المكوّنات المشتركة ← شاشة الترحيب نفسها.

pubspec.yaml — الحزم والأصول

pubspec.yaml

أُضيفت هذه الأسطر بالأمر `flutter pub add` ثم عُدّل قسم الأصول يدويًا.

السطر 37

```
37    flutter_svg: ^2.3.0
```

ماذا يفعل؟ يضيف حزمة `flutter_svg` لعرض ملفات SVG.

لماذا بهذه الطريقة؟ الأيقونات والشعار والتطريز مُصدَّرة من Figma بصيغة SVG. SVG يبقى حادًّا على أي حجم شاشة وأخف من الصور النقطية، و`Image.asset` في Flutter لا يعرض SVG.

السطر 38

```
38    google_fonts: ^8.2.1
```

ماذا يفعل؟ يضيف حزمة `google_fonts` لاستخدام خط Cairo.

لماذا بهذه الطريقة؟ كل نصوص التصميم بخط Cairo. الحزمة تجلب الخط وتخزّنه، فلا نحتاج نسخ ملفات الخط الآن. (لتطبيق يعمل دون إنترنت يُفضّل لاحقًا تضمين الخط داخل الأصول.)

الأسطر 39–40

```
39    flutter_localizations:
40      sdk: flutter
```

ماذا يفعل؟ يضيف `flutter_localizations` من الـ SDK نفسه.

لماذا بهذه الطريقة؟ مطلوبة لتعريب مكوّنات Material (اتجاه RTL، أسماء الأزرار، لوحة المفاتيح، التقويم). بدونها يبقى الاتجاه LTR في بعض المكوّنات.

السطر 62

```
62    uses-material-design: true
```

ماذا يفعل؟ يفعّل خط أيقونات Material.

لماذا بهذه الطريقة؟ سطر افتراضي من قالب المشروع؛ نبقيه لأن بعض مكوّنات Material تستخدمه داخليًا.

الأسطر 64–66

```
64    assets:
65      - assets/images/brand/
66      - assets/icons/
```

ماذا يفعل؟ يسجّل المجلدين `assets/images/brand/` و `assets/icons/` كأصول.

لماذا بهذه الطريقة؟ Flutter لا يرى أي ملف إلا إذا سُجّل هنا. كتابة المجلد بشرطة مائلة في النهاية تضيف كل ملفاته دفعة واحدة، فلا نعدّل pubspec مع كل أيقونة جديدة.

lib/main.dart — نقطة الدخول

lib/main.dart

السطر 1

```
1    import 'package:flutter/material.dart';
```

ماذا يفعل؟ يستورد مكتبة Material.

لماذا بهذه الطريقة؟ فيها `runApp` والودجتات الأساسية.

السطر 3

```
3    import 'app/rizma_app.dart';
```

ماذا يفعل؟ يستورد ملف التطبيق `RizmaApp` .

لماذا بهذه الطريقة؟ نفصل «التطبيق» عن «نقطة الدخول» ليبقى `main` قصيرًا وسهل الاختبار.

الأسطر 5–7

```
5    void main() {
6      runApp(const RizmaApp());
7    }
```

ماذا يفعل؟ الدالة `main` تشغّل التطبيق بـ `runApp(const RizmaApp())` .

لماذا بهذه الطريقة؟ كان القالب الافتراضي يضع ودجت `MyApp` هنا؛ استبدلناه ونقلنا التطبيق إلى `app/rizma_app.dart` لتبقى نقطة الدخول مجرد تشغيل. `const` يوفّر إعادة بناء غير ضرورية.

lib/app/rizma_app.dart — جذر التطبيق

lib/app/rizma_app.dart

الأسطر 1–2

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_localizations/flutter_localizations.dart';
```

ماذا يفعل؟ استيراد Material وحزمة التعريب.

لماذا بهذه الطريقة؟ `flutter_localizations` تعطي «المندوبين» (delegates) الذين يترجمون مكوّنات Flutter.

الأسطر 4–5

```
4  import '../core/theme/app_theme.dart';
5  import '../features/welcome/welcome_screen.dart';
```

ماذا يفعل؟ استيراد الثيم وشاشة الترحيب.

لماذا بهذه الطريقة؟ المسارات نسبية (`../`) لأن الملف داخل `app/` والثيم داخل `core/`.

الأسطر 7–8

```
7  class RizmaApp extends StatelessWidget {
8    const RizmaApp({super.key});
```

ماذا يفعل؟ ودجت `RizmaApp` بلا حالة (`StatelessWidget`) مع مفتاح `key`.

لماذا بهذه الطريقة؟ جذر التطبيق لا يتغيّر أثناء التشغيل، فلا حاجة لـ `StatefulWidget`.

الأسطر 10–12

```
10   @override
11   Widget build(BuildContext context) {
12     return MaterialApp(
```

ماذا يفعل؟ يبني `MaterialApp`.

لماذا بهذه الطريقة؟ `MaterialApp` هو الودجت الذي يوفّر الملاحة (`Navigator`) والثيم والتعريب لكل الشاشات تحته.

السطر 13

```
13       title: 'رزمة',
```

ماذا يفعل؟ عنوان التطبيق `رزمة`.

لماذا بهذه الطريقة؟ يظهر في مبدّل المهام لنظام التشغيل.

السطر 14

```
14       debugShowCheckedModeBanner: false,
```

ماذا يفعل؟ يخفي شريط «DEBUG».

لماذا بهذه الطريقة؟ نظافة بصرية في لقطات الشاشة وأثناء التطوير.

السطر 15

```
15        theme: AppTheme.light,
```

ماذا يفعل؟ يربط الثيم `AppTheme.light` .

لماذا بهذه الطريقة؟ كل الألوان والخطوط تأتي من مكان واحد بدل تكرارها في كل شاشة.

الأسطر 16–17

```
16        locale: const Locale('ar'),
17        supportedLocales: const [Locale('ar')],
```

ماذا يفعل؟ يحدد اللغة `ar` ويحصر اللغات المدعومة في العربية.

لماذا بهذه الطريقة؟ هذا السطر هو ما يقلب كل التطبيق إلى RTL تلقائيًا: لا نحتاج `Directionality` يدوي ولا قلب الصفوف في كل شاشة.

الأسطر 18–22

```
18        localizationsDelegates: const [
19          GlobalMaterialLocalizations.delegate,
20          GlobalWidgetsLocalizations.delegate,
21          GlobalCupertinoLocalizations.delegate,
22        ],
```

ماذا يفعل؟ قائمة مندوبي التعريب الثلاثة: Material وWidgets وCupertino.

لماذا بهذه الطريقة؟ Widgets يحدد اتجاه النص، وMaterial يترجم مكوّنات مثل حقل النص ومنتقي التاريخ، وCupertino لمكوّنات iOS. تركها ناقصة يسبب أخطاء أو اتجاهًا خاطئًا.

السطر 23

```
23        home: const WelcomeScreen(),
```

ماذا يفعل؟ الشاشة الأولى هي `WelcomeScreen` .

لماذا بهذه الطريقة؟ التصميم يبدأ بشاشة الترحيب. `home` كافٍ حاليًا؛ سننتقل إلى مسارات مسمّاة (GoRouter) عندما تكثر الشاشات.

الأسطر 24–26

```
24      );
25    }
26  }
```

ماذا يفعل؟ إغلاق `MaterialApp` والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/theme/app_colors.dart — ألوان التصميم

`lib/core/theme/app_colors.dart`

كل قيمة هنا منسوخة من متغيّرات `Color/...` (Figma) بنفس الاسم تقريبًا حتى يسهل تتبّعها. الصيغة `0xFF` تعني «معتم بالكامل» ثم ستة أرقام سداسية عشرية للون.

السطر 1

```dart
import 'package:flutter/painting.dart';
```

ماذا يفعل؟ يستورد `Color` فقط من مكتبة `painting`.

لماذا بهذه الطريقة؟ الملف لا يحتاج كل Material، فنستورد الأخف.

الأسطر 3–4

```dart
/// ألوان التصميم كما هي في Figma (Color/*).
abstract final class AppColors {
```

ماذا يفعل؟ صنف `AppColors` بمُعرّف `abstract final`.

لماذا بهذه الطريقة؟ `abstract` يمنع إنشاء كائن منه، و`final` يمنع الوراثة منه؛ هو مجرد حاوية ثوابت. لا نكتب `Color(0xFF...)` متفرقة في الشاشات فنغيّر اللون من مكان واحد.

الأسطر 5–11

```dart
  // الخلفيات والأسطح
  static const ground = Color(0xFFF4F5F1);
  static const light = Color(0xFFF4F1E8);
  static const card = Color(0xFFFFFFFF);
  static const mute = Color(0xFFE6EAE5);
  static const line = Color(0xFFE1E6E2);
  static const disabled = Color(0xFFC9D2CD);
```

ماذا يفعل؟ ألوان الخلفيات والأسطح: `ground` خلفية الشاشات، `light` لون النص على الداكن، `card` أبيض البطاقات، `mute` رمادي فاتح للأزرار الثانوية وأشرطة التقدّم، `line` الحدود، `disabled` للمعطّل.

لماذا بهذه الطريقة؟ التسمية بحسب الدور لا بحسب اللون (مثل `line` لا `lightGrey`) حتى لو تغيّر اللون لاحقًا يبقى الاسم صحيحًا.

الأسطر 13–15

```dart
  // النصوص
  static const ink = Color(0xFF0F2A2E);
  static const soft = Color(0xFF55655F);
```

ماذا يفعل؟ ألوان النص: `ink` للنص الأساسي الداكن، و`soft` للنص الثانوي الباهت.

لماذا بهذه الطريقة؟ درجتان فقط تكفيان لتسلسل بصري واضح وتحافظان على تباين مقروء.

الأسطر 17–21

```dart
  // الأخضر (اللون الأساسي)
  static const teal = Color(0xFF177A63);
  static const tealDark = Color(0xFF0E5C4A);
  static const tealTint = Color(0xFFDCEFE6);
  static const onTeal = Color(0xFFD5EEE3);
```

ماذا يفعل؟ عائلة الأخضر: اللون الأساسي `teal` وأغمق `tealDark` وفاتح `tealTint` والنص على الأخضر `onTeal`.

لماذا بهذه الطريقة؟ نمط `Tint` للخلفيات الناعمة خلف الأيقونات، ونمط `Dark` لمسارات أشرطة التقدّم على الأخضر.

الأسطر 23–27

```
    // العنبري
    static const amber = Color(0xFFE9A23B);
    static const amberDark = Color(0xFF8F540A);
    static const amberTint = Color(0xFFFCEFD6);
    static const amberLine = Color(0xFFF1D29C);
```

ماذا يفعل؟ عائلة العنبري: `amber` , `amberDark` , `amberTint` , `amberLine` .

لماذا بهذه الطريقة؟ تُستخدم لبطاقة «يحتاج مراجعة» والنجوم. `amberLine` لون الحد حولها.

الأسطر 29–31

```
    // الأزرق
    static const blue = Color(0xFF2F66C9);
    static const blueTint = Color(0xFFDEE7F8);
```

ماذا يفعل؟ عائلة الأزرق.

لماذا بهذه الطريقة؟ للعلوم وملف الجهاز دون إنترنت.

الأسطر 33–35

```
    // المرجاني
    static const coral = Color(0xFFB94F2A);
    static const coralTint = Color(0xFFF8E1D7);
```

ماذا يفعل؟ عائلة المرجاني.

لماذا بهذه الطريقة؟ للّغة العربية وسلسلة الأيام المتتالية.

الأسطر 37–40

```
    // البنفسجي
    static const purple = Color(0xFF6A4FB3);
    static const purpleTint = Color(0xFFE9E3F6);
}
```

ماذا يفعل؟ عائلة البنفسجي ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ للإنجليزية وتحدّي اليوم. كل مادة لها لونها الخاص لتمييزها بسرعة.

lib/core/theme/app_text_styles.dart — أنماط النص

`lib/core/theme/app_text_styles.dart`

أسماء الأنماط تحمل الحجم (مثل `heading22`) لتتطابق مع أسماء الأنماط في Figma (Heading/22, Body/16...).

الأسطر 1–2

```dart
import 'package:flutter/painting.dart';
import 'package:google_fonts/google_fonts.dart';
```

ماذا يفعل؟ استيراد `painting` وحزمة `google_fonts`.

لماذا بهذه الطريقة؟ نحتاج `TextStyle` و`FontWeight` و`GoogleFonts.cairo`.

السطر 4

```dart
import 'app_colors.dart';
```

ماذا يفعل؟ استيراد الألوان.

لماذا بهذه الطريقة؟ كل نمط يحمل لونه الافتراضي من التصميم.

الأسطر 6–8

```dart
/// أنماط النص كما في Figma – كلها بخط Cairo.
/// الوزن: SemiBold=600، Bold=700، ExtraBold=800.
abstract final class AppTextStyles {
```

ماذا يفعل؟ تعليق توضيحي وبداية الصنف `AppTextStyles`.

لماذا بهذه الطريقة؟ حاوية ثوابت كما في الألوان. تعليق الأوزان يربط أسماء Figma (SemiBold...) بأرقام Flutter (600...).

الأسطر 9–21

```dart
  static TextStyle _cairo(
    double size,
    FontWeight weight,
    double height,
    Color color,
  ) {
    return GoogleFonts.cairo(
      fontSize: size,
      fontWeight: weight,
      height: height,
      color: color,
    );
  }
```

ماذا يفعل؟ دالة خاصة `_cairo` تبني `TextStyle` بخط Cairo: الحجم، الوزن، الارتفاع (height)، اللون.

لماذا بهذه الطريقة؟ منعًا لتكرار `GoogleFonts.cairo(...)` عشرين مرة. `height` في Flutter مضاعف لحجم الخط، وهو يطابق `line-height` غير المقيّد بوحدة في Figma (مثل 1.5) فتتطابق مسافات الأسطر مع التصميم.

الأسطر 23–29

```
23    // الشعار والعناوين
24    static final wordmark40 = _cairo(40, FontWeight.w800, 1.3, AppColors.light);
25    static final tagline18 = _cairo(18, FontWeight.w700, 1.6, AppColors.light);
26    static final title26 = _cairo(26, FontWeight.w800, 1.35, AppColors.ink);
27    static final heading22 = _cairo(22, FontWeight.w800, 1.35, AppColors.ink);
28    static final cardTitle20 = _cairo(20, FontWeight.w800, 1.4, AppColors.card);
29    static final section18 = _cairo(18, FontWeight.w800, 1.4, AppColors.ink);
```

ماذا يفعل؟ أنماط الشعار والعناوين: `wordmark40`، `tagline18`، `title26`، `heading22`، `cardTitle20`، `section18`.

لماذا بهذه الطريقة؟ الأوزان 800 (ExtraBold) للعناوين و700 للشعار الفرعي حسب Figma. `static final` لا `const` لأن `GoogleFonts.cairo` ليست دالة ثابتة وقت الترجمة.

الأسطر 31–35

```
31    // النص الأساسي
32    static final body16Strong = _cairo(16, FontWeight.w800, 1.5, AppColors.ink);
33    static final body15 = _cairo(15, FontWeight.w700, 1.5, AppColors.ink);
34    static final label14 = _cairo(14, FontWeight.w700, 1.45, AppColors.soft);
35    static final caption13 = _cairo(13, FontWeight.w600, 1.45, AppColors.soft);
```

ماذا يفعل؟ أنماط النص الأساسي: `body16Strong`، `body15`، `label14`، `caption13`.

لماذا بهذه الطريقة؟ ثلاثة أحجام مقروءة (13/14/15/16) تُغطي معظم الواجهة.

الأسطر 36–41

```
36    static final captionBold13 = _cairo(
37      13,
38      FontWeight.w700,
39      1.45,
40      AppColors.soft,
41    );
```

ماذا يفعل؟ نمط `captionBold13` (13 بوزن 700).

لماذا بهذه الطريقة؟ مستقل عن `caption13` (وزن 600) لأن التصميم يستخدم الوزنين في أماكن مختلفة (الشريط السفلي، بطاقة المراجعة).

السطر 42

```
42    static final percent12 = _cairo(12, FontWeight.w800, 1.4, AppColors.ink);
```

ماذا يفعل؟ نمط `percent12` لنسب التقدّم.

لماذا بهذه الطريقة؟ خط صغير سميك لرقم قصير فيُقرأ بسهولة.

الأسطر 44–49

```
44    // مكوّنات محدّدة
45    static final button16 = _cairo(16, FontWeight.w800, 1.5, AppColors.card);
46    static final input17 = _cairo(17, FontWeight.w700, 1.5, AppColors.ink);
47    static final gradeNumber24 = _cairo(24, FontWeight.w800, 1.3, AppColors.ink);
48    static final chip13 = _cairo(13, FontWeight.w700, 1.5, AppColors.ink);
49  }
```

ماذا يفعل؟ أنماط مكوّنات: `button16`, `input17`, `gradeNumber24`, `chip13`، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ مرتبطة بمكوّنات محدّدة. حين يحتاج مكوّن لونًا آخر نستخدم `copyWith(color: ...)` بدل إنشاء نمط جديد لكل لون.

lib/core/theme/app_theme.dart — الثيم العام

`lib/core/theme/app_theme.dart`

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2   import 'package:google_fonts/google_fonts.dart';
3
4   import 'app_colors.dart';
```

ماذا يفعل؟ استيرادات Material وGoogleFonts والألوان.

لماذا بهذه الطريقة؟ ما يلزم لبناء `ThemeData`.

الأسطر 6–7

```
6   abstract final class AppTheme {
7     static ThemeData get light {
```

ماذا يفعل؟ صنف `AppTheme` وخاصية ثابتة `light` ترجع `ThemeData`.

لماذا بهذه الطريقة؟ خاصية (getter) بدل ثابت لأن `GoogleFonts.cairoTextTheme()` تُستدعى وقت التشغيل. اسمها `light` لأن الوضع الداكن غير موجود في التصميم حاليًا، ويمكن إضافة `dark` لاحقًا بنفس الشكل.

الأسطر 8–17

```
8       final colorScheme =
9           ColorScheme.fromSeed(
10            seedColor: AppColors.teal,
11            brightness: Brightness.light,
12          ).copyWith(
13            primary: AppColors.teal,
14            onPrimary: AppColors.card,
15            surface: AppColors.ground,
16            onSurface: AppColors.ink,
17          );
```

ماذا يفعل؟ يولّد `ColorScheme` من لون البذرة `teal` ثم يثبّت بعض الألوان يدويًا: `primary`, `onPrimary`, `surface`, `onSurface`.

لماذا بهذه الطريقة؟ `fromSeed` يعطي لوحة كاملة متناسقة لمكوّنات Material التي لا نخصّصها (مثل مؤشر النص). ثم نفرض ألواننا من Figma حتى لا يعدّل Material اللون الأخضر تلقائيًا.

الأسطر 19–20

```
19      return ThemeData(
20        useMaterial3: true,
```

ماذا يفعل؟ يبني `ThemeData` ويفعّل Material 3.

لماذا بهذه الطريقة؟ Material 3 هو الافتراضي الحديث وتتوافق معه مكوّنات Flutter الحالية.

السطر 21

```
21        colorScheme: colorScheme,
```

ماذا يفعل؟ يمرّر `colorScheme`.

لماذا بهذه الطريقة؟ مصدر الألوان للمكوّنات الجاهزة.

السطر 22

```
22        scaffoldBackgroundColor: AppColors.ground,
```

ماذا يفعل؟ لون خلفية كل `Scaffold` هو `ground`.

لماذا بهذه الطريقة؟ حتى لا نكرر `backgroundColor` في كل شاشة.

السطر 23

```
23        textTheme: GoogleFonts.cairoTextTheme(),
```

ماذا يفعل؟ يجعل خط Cairo هو خط التطبيق الافتراضي.

لماذا بهذه الطريقة؟ أي `Text` بلا نمط (مثل نص زر `TextButton`) يستخدم Cairo بدل خط النظام.

الأسطر 24–26

```
24      );
25    }
26  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة والصنف.

lib/core/constants/app_assets.dart — مسارات الأصول

lib/core/constants/app_assets.dart

الأسطر 1–2

```
1   /// مسارات الأصول (SVG) المصدَّرة من Figma — مكان واحد لكل المسارات.
2   abstract final class AppAssets {
```

ماذا يفعل؟ تعليق وصنف `AppAssets` (حاوية ثوابت).

لماذا بهذه الطريقة؟ نفس فكرة `AppColors`: مكان واحد لكل مسارات الملفات.

الأسطر 3–4

```
3     static const _brand = 'assets/images/brand';
4     static const _icons = 'assets/icons';
```

ماذا يفعل؟ ثابتان خاصان للمجلدين `_brand` و `_icons`.

لماذا بهذه الطريقة؟ نكتب جذر المجلد مرة واحدة؛ لو غيّرنا اسم المجلد نعدّل سطرًا واحدًا.

الأسطر 6–11

```
6     // الهوية
7     static const tatreezBand = '$_brand/tatreez_band.svg';
8     static const mascot150 = '$_brand/mascot_150.svg';
9     static const mascot56 = '$_brand/mascot_56.svg';
10    static const mascot40 = '$_brand/mascot_40.svg';
11    static const logoOnDark = '$_brand/logo_on_dark.svg';
```

ماذا يفعل؟ أصول الهوية: شريط التطريز، الشخصية بثلاثة أحجام (150/56/40)، والشعار على الداكن.

لماذا بهذه الطريقة؟ الشخصية مُصدَّرة من Figma بثلاثة ملفات لأن كل حجم مُصدَّر بأبعاده وعرضه الخاصة؛ نستعمل كل ملف في مكانه دون تكبير أو تصغير.

الأسطر 13–17

```
13    // شاشة الترحيب
14    static const bookTile = '$_icons/book_tile.svg';
15    static const cloudOff = '$_icons/cloud_off.svg';
16    static const target = '$_icons/target.svg';
17    static const fwdOnboarding = '$_icons/fwd_onboarding.svg';
```

ماذا يفعل؟ أيقونات شاشة الترحيب.

لماذا بهذه الطريقة؟ مجمّعة بحسب الشاشة لتسهيل إيجادها وحذفها.

الأسطر 19–21

```
19    // شاشة اختيار الصف
20    static const back = '$_icons/back.svg';
21    static const fwdGrade = '$_icons/fwd_grade.svg';
```

ماذا يفعل؟ أيقونتا الرجوع والمتابعة لشاشة اختيار الصف.

لماذا بهذه الطريقة؟ أيقونة «رجوع» مرسومة في Figma بالاتجاه الصحيح لـ RTL (سهم لليمين)، فلا نقلبها برمجيًا.

الأسطر 23–27

```
23    // الشريط السفلي
24    static const navUser = '$_icons/nav_user.svg';
25    static const navChart = '$_icons/nav_chart.svg';
26    static const navBook = '$_icons/nav_book.svg';
27    static const navHome = '$_icons/nav_home.svg';
```

ماذا يفعل؟ أيقونات الشريط السفلي.

لماذا بهذه الطريقة؟ تُستخدم في الواجهة 3.

الأسطر 29–41

```
  // الشاشة الرئيسية
  static const starFilled = '$_icons/star_filled.svg';
  static const flame = '$_icons/flame.svg';
  static const calcCard = '$_icons/calc_card.svg';
  static const play = '$_icons/play.svg';
  static const warn = '$_icons/warn.svg';
  static const fwdRow = '$_icons/fwd_row.svg';
  static const bolt = '$_icons/bolt.svg';
  static const leaf = '$_icons/leaf.svg';
  static const chat = '$_icons/chat.svg';
  static const bookSubject = '$_icons/book_subject.svg';
  static const calcSubject = '$_icons/calc_subject.svg';
}
```

ماذا يفعل؟ أيقونات الشاشة الرئيسية ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ نبقي أسماء المسارات وصفية (مثل `calcCard` و`calcSubject`) لأن الأيقونة نفسها مُصدَّرة بحجمين مختلفين.

lib/core/widgets/icon_tile.dart — مربع الأيقونة الملوّن

lib/core/widgets/icon_tile.dart

مكوّن يتكرر في ٤ أماكن (الترحيب، المراجعة، التحدّي، المواد) فاستحق ودجتًا مستقلًّا.

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
```

ماذا يفعل؟ استيراد Material وSVG.

لماذا بهذه الطريقة؟ الأيقونة تُرسم بـ `SvgPicture`.

الأسطر 4–5

```
/// مربع ملوّن بزوايا مدوّرة يحوي أيقونة في الوسط.
class IconTile extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `IconTile`.

لماذا بهذه الطريقة؟ ودجت بلا حالة.

الأسطر 6–12

```
  const IconTile({
    super.key,
    required this.asset,
    required this.color,
    this.size = 48,
    this.iconSize = 24,
  });
```

ماذا يفعل؟ الباني: `asset` و`color` مطلوبان، و`size` افتراضي 48 و`iconSize` 24.

لماذا بهذه الطريقة؟ القيم الافتراضية هي الأكثر استخدامًا في التصميم (48/24)، ومكوّنات أخرى تمرّر 44/22.

الأسطر 14–17

```
14    final String asset;
15    final Color color;
16    final double size;
17    final double iconSize;
```

ماذا يفعل؟ الحقول: مسار الأيقونة، لون الخلفية، حجم المربع، حجم الأيقونة.

لماذا بهذه الطريقة؟ كلها `final` لأن الودجت غير قابل للتغيير بعد إنشائه.

الأسطر 19–20

```
19    @override
20    Widget build(BuildContext context) {
```

ماذا يفعل؟ دالة `build`.

لماذا بهذه الطريقة؟ تعيد شجرة الودجت.

الأسطر 21–30

```
21      return Container(
22        width: size,
23        height: size,
24        decoration: BoxDecoration(
25          color: color,
26          borderRadius: BorderRadius.circular(14),
27        ),
28        alignment: Alignment.center,
29        child: SvgPicture.asset(asset, width: iconSize, height: iconSize),
30      );
```

ماذا يفعل؟ `Container` بعرض وارتفاع `size`، خلفية ملوّنة وزوايا نصف قطرها 14، والأيقونة في المنتصف بحجم `iconSize`.

لماذا بهذه الطريقة؟ نصف القطر 14 مأخوذ من Figma. `alignment: center` يضع الأيقونة في وسط المربع. نحدّد حجم الأيقونة صراحةً (لا نتركه يتمدد) كما يشترط التصدير من Figma.

الأسطر 31–32

```
31    }
32  }
```

ماذا يفعل؟ إغلاق الدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/widgets/primary_button.dart — الزر الرئيسي

`lib/core/widgets/primary_button.dart`

يُستخدم في الشاشات الثلاث: «ابدأ» و«متابعة · الصف 5» و«متابعة التعلم».

الأسطر 1–5

```dart
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
```

ماذا يفعل؟ استيرادات Material وSVG والألوان وأنماط النص.

لماذا بهذه الطريقة؟ ما يحتاجه الزر لرسم نفسه.

الأسطر 7–8

```dart
/// الزر الرئيسي العريض. إذا كان [onPressed] فارغًا يظهر معطّلًا.
class PrimaryButton extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `PrimaryButton`.

لماذا بهذه الطريقة؟ ودجت بلا حالة؛ الحالة (مفعّل/معطّل) تأتي من الخارج عبر `onPressed`.

الأسطر 9–17

```dart
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.iconAsset,
    this.height = 52,
    this.backgroundColor = AppColors.teal,
    this.foregroundColor = AppColors.card,
  });
```

ماذا يفعل؟ الباني: `label` و`onPressed` مطلوبان، والباقي اختياري بقيم افتراضية من التصميم (ارتفاع 52، خلفية teal، نص أبيض).

لماذا بهذه الطريقة؟ تمرير `onPressed: null` معناه «معطّل» — هذا هو العرف في Flutter (مثل `ElevatedButton`) فيفهمه أي مطوّر.

الأسطر 19–24

```dart
  final String label;
  final VoidCallback? onPressed;
  final String? iconAsset;
  final double height;
  final Color backgroundColor;
  final Color foregroundColor;
```

ماذا يفعل؟ الحقول. `iconAsset` و`onPressed` قابلان لأن يكونا `null`.

لماذا بهذه الطريقة؟ `VoidCallback?` تسمح بحالة التعطيل. الأيقونة اختيارية لأن بعض الأزرار بلا أيقونة.

الأسطر 26–28

```
26    @override
27    Widget build(BuildContext context) {
28      final enabled = onPressed != null;
```

ماذا يفعل؟ يحسب `enabled` من كون `onPressed` غير فارغ.

لماذا بهذه الطريقة؟ مصدر واحد للحقيقة يُستخدم للّون وللوصول (semantics).

الأسطر 30–34

```
30      return Semantics(
31        button: true,
32        enabled: enabled,
33        label: label,
34        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` تعرّف الزر لقارئ الشاشة: هو زر، مفعّل أو لا، اسمه `label`، ونتجاهل الودجتات الداخلية.

لماذا بهذه الطريقة؟ يسمعه مستخدم TalkBack/VoiceOver «ابدأ، زر» مرة واحدة بدل قراءة النص والأيقونة منفصلين. هذا من متطلبات إمكانية الوصول.

الأسطر 35–38

```
35        child: Material(
36          color: enabled ? backgroundColor : AppColors.disabled,
37          borderRadius: BorderRadius.circular(16),
38          clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` يعطي الخلفية الملونة (teal، أو رمادي إن كان معطّلًا) وزوايا 16 وقصًّا بحواف ناعمة.

لماذا بهذه الطريقة؟ `InkWell` يرسم تموّج اللمس على أقرب `Material` فقط؛ لو لوّنا `Container` بدل `Material` لاختفى التموّج خلف اللون. `Clip.antiAlias` يمنع التموّج من الخروج خارج الزوايا المدوّرة.

الأسطر 39–40

```
39          child: InkWell(
40            onTap: onPressed,
```

ماذا يفعل؟ `InkWell` يلتقط الضغط ويستدعي `onPressed`.

لماذا بهذه الطريقة؟ عندما تكون `null` يتعطل اللمس تلقائيًا.

الأسطر 41–44

```
41            child: Container(
42              height: height,
43              padding: const EdgeInsets.symmetric(horizontal: 20),
44              alignment: Alignment.center,
```

ماذا يفعل؟ `Container` بارتفاع `height` وحشوة أفقية 20 ومحاذاة للمنتصف.

لماذا بهذه الطريقة؟ الارتفاع ثابت (52/48) كما في Figma ليبقى الزر بنفس الحجم مهما كان طول النص.

الأسطر 45–48

```
            child: Row(
              mainAxisSize: MainAxisSize.min,
              spacing: 8,
              children: [
```

ماذا يفعل؟ `Row` بحجم أدنى (`min`) ومسافة 8 بين الأيقونة والنص.

لماذا بهذه الطريقة؟ `MainAxisSize.min` يجعل الصف بقدر محتواه فيتوسّط داخل الـ Container. خاصية `spacing` (من Flutter 3.27) تغني عن وضع `SizedBox` بين كل عنصرين.

الأسطر 49–50

```
                if (iconAsset != null)
                  SvgPicture.asset(iconAsset!, width: 20, height: 20),
```

ماذا يفعل؟ إن وُجدت أيقونة تُرسم بحجم 20.

لماذا بهذه الطريقة؟ الأيقونة هي أول عنصر في الصف: في RTL يعني أنها على اليمين من النص، كما في تصميم Figma.

الأسطر 51–56

```
                Text(
                  label,
                  style: AppTextStyles.button16.copyWith(
                    color: foregroundColor,
                  ),
                ),
```

ماذا يفعل؟ نص الزر بنمط `button16` مع لون `foregroundColor` .

لماذا بهذه الطريقة؟ `copyWith` يغيّر اللون فقط (أبيض على الأخضر، أخضر على الأبيض).

الأسطر 57–64

```
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة والصنف.

lib/features/welcome/welcome_screen.dart — شاشة «مرحبًا»

`lib/features/welcome/welcome_screen.dart`

الإطار في (844×390) `مرحبًا 01` :Figma. تتكوّن من قسمين: «هيرو» داكن في الأعلى بزوايا سفلية مدوّرة، وقسم محتوى فيه ثلاث نقاط قيمة ونقاط صفحات وزر.

الأسطر 1–3

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_svg/flutter_svg.dart';
```

ماذا يفعل؟ استيراد Material وخدمات النظام وSVG.

لماذا بهذه الطريقة؟ `services.dart` فيها `SystemUiOverlayStyle` لتلوين أيقونات شريط الحالة.

الأسطر 5–10

```dart
import '../../core/constants/app_assets.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/primary_button.dart';
import '../grade_selection/grade_selection_screen.dart';
```

ماذا يفعل؟ استيراد الأصول والألوان والأنماط والمكوّنات المشتركة وشاشة اختيار الصف.

لماذا بهذه الطريقة؟ نستورد الشاشة التالية لأن زر «ابدأ» ينتقل إليها.

الأسطر 12–14

```dart
/// الشاشة 01 – مرحبًا
class WelcomeScreen extends StatelessWidget {
  const WelcomeScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `WelcomeScreen` بلا حالة.

لماذا بهذه الطريقة؟ الشاشة ثابتة؛ لا تحتفظ بأي بيانات.

الأسطر 16–20

```dart
  void _start(BuildContext context) {
    Navigator.of(context).push(
      MaterialPageRoute<void>(builder: (_) => const GradeSelectionScreen()),
    );
  }
```

ماذا يفعل؟ الدالة `_start` تدفع شاشة اختيار الصف فوق الحالية بـ `Navigator.push`.

لماذا بهذه الطريقة؟ `push` وليس `pushReplacement` لأن المستخدم قد يرجع من خطوة الصف إلى الترحيب بزر الرجوع.
`MaterialPageRoute<void>` لأن الشاشة لا ترجع قيمة.

الأسطر 22–23

```dart
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ دالة `build`.

لماذا بهذه الطريقة؟ تعيد شجرة الواجهة.

الأسطر 24–25

```
24      return AnnotatedRegion<SystemUiOverlayStyle>(
25        value: SystemUiOverlayStyle.light,
```

ماذا يفعل؟ `AnnotatedRegion` يجعل أيقونات شريط الحالة فاتحة.

لماذا بهذه الطريقة؟ الهيرو داكن، فلو بقيت الساعة والبطارية بلون داكن لاختفت. `SystemUiOverlayStyle.light` = أيقونات بيضاء.

السطر 26

```
26        child: Scaffold(
```

ماذا يفعل؟ `Scaffold` هيكل الصفحة.

لماذا بهذه الطريقة؟ يأخذ لون الخلفية `ground` من الثيم.

السطر 27

```
27          body: SingleChildScrollView(
```

ماذا يفعل؟ `SingleChildScrollView` يجعل المحتوى قابلًا للتمرير.

لماذا بهذه الطريقة؟ الشاشة بطول 844 في التصميم، لكن الأجهزة الصغيرة أو الخط الكبير قد تتجاوز ذلك؛ التمرير يمنع خطأ «overflow».

الأسطر 28–31

```
28            child: Column(
29              crossAxisAlignment: CrossAxisAlignment.stretch,
30              children: [
31                const _Hero(),
```

ماذا يفعل؟ `Column` يمدّ أبناءه بعرض الشاشة ويضع الهيرو أولًا.

لماذا بهذه الطريقة؟ `CrossAxisAlignment.stretch` يعطي الهيرو العرض الكامل.

الأسطر 32–36

```
32                Padding(
33                  padding: const EdgeInsets.symmetric(
34                    horizontal: 24,
35                    vertical: 28,
36                  ),
```

ماذا يفعل؟ حشوة للمحتوى: 24 أفقيًا و28 عموديًا.

لماذا بهذه الطريقة؟ القيم من Figma.

الأسطر 37–40

```
37                child: Column(
38                  crossAxisAlignment: CrossAxisAlignment.stretch,
39                  spacing: 18,
40                  children: [
```

ماذا يفعل؟ `Column` داخلي يمدّ العناصر مع مسافة 18 بينها.

لماذا بهذه الطريقة؟ خاصية `spacing` تعني ألا نكرر `SizedBox(height: 18)` بين كل عنصرين.

الأسطر 41–46

```
41                    const _ValuePoint(
42                      asset: AppAssets.bookTile,
43                      tileColor: AppColors.tealTint,
44                      title: 'رزمتك… بطريقة أوضح',
45                      subtitle: 'شرح، ملخص، فيديو وتدريب لكل درس.',
46                    ),
```

ماذا يفعل؟ نقطة القيمة الأولى: أيقونة كتاب، خلفية `tealTint`، العنوان والوصف.

لماذا بهذه الطريقة؟ ثلاث نقاط قيمة بنفس التركيب فصنعنا لها ودجتًا `_ValuePoint` وغذّيناها بالبيانات. `const` لأن كل قيمها ثابتة.

الأسطر 47–52

```
47                    const _ValuePoint(
48                      asset: AppAssets.cloudOff,
49                      tileColor: AppColors.blueTint,
50                      title: 'تعلّم دون إنترنت',
51                      subtitle: 'نزّل الدرس مرة وتعلّم في أي وقت.',
52                    ),
```

ماذا يفعل؟ النقطة الثانية: السحابة المشطوبة بخلفية زرقاء فاتحة (التعلّم دون إنترنت).

لماذا بهذه الطريقة؟ اللون الأزرق يميّز ميزة «دون اتصال».

الأسطر 53–58

```
53                    const _ValuePoint(
54                      asset: AppAssets.target,
55                      tileColor: AppColors.amberTint,
56                      title: 'اعرف أين تحتاج مراجعة',
57                      subtitle: 'خطة قصيرة مبنية على نتيجتك.',
58                    ),
```

ماذا يفعل؟ النقطة الثالثة: الهدف بخلفية عنبرية (المراجعة).

لماذا بهذه الطريقة؟ اللون العنبري يعبّر عن «انتباه/مراجعة».

السطر 59

```
59                    const _PageDots(count: 3, current: 0),
```

ماذا يفعل؟ نقاط الصفحات: 3 نقاط والحالية الأولى.

لماذا بهذه الطريقة؟ التصميم يوحي بأن الترحيب مكوّن من 3 شاشات تعريفية؛ الودجت جاهز لذلك عبر `count` و`current`.

الأسطر 60–64

```
                    PrimaryButton(
                      label: 'ابدأ',
                      iconAsset: AppAssets.fwdOnboarding,
                      onPressed: () => _start(context),
                    ),
```

ماذا يفعل؟ زر «ابدأ» بأيقونة سهم، ينفّذ `_start`.

لماذا بهذه الطريقة؟ نستخدم `PrimaryButton` المشترك بدل رسم زر جديد. الدالة المجهولة `() => _start(context)` لأننا نحتاج تمرير `context`.

الأسطر 65–72

```
                    TextButton(
                      onPressed: () {},
                      style: TextButton.styleFrom(
                        foregroundColor: AppColors.teal,
                        textStyle: AppTextStyles.body15,
                      ),
                      child: const Text('لديّ ملف طالب على هذا الجهاز'),
                    ),
```

ماذا يفعل؟ زر نصي «لديّ ملف طالب على هذا الجهاز» بلون teal.

لماذا بهذه الطريقة؟ `TextButton` لأنه إجراء ثانوي لا يستحق ثِقل الزر الرئيسي. المعالج `() {}` فارغ حاليًا لأن شاشة استيراد الملف غير موجودة في أول 3 واجهات. أبقينا حجم اللمس الافتراضي (48) لإمكانية الوصول.

الأسطر 73–82

```
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة والصنف.

الأسطر 84–87

```
class _Hero extends StatelessWidget {
  const _Hero();

  @override
```

ماذا يفعل؟ ودجت `_Hero` خاص (بشرطة سفلية) بلا حالة.

لماذا بهذه الطريقة؟ الشرطة السفلية تجعله خاصًّا بالملف فلا يظهر خارجه. فصلناه لأن قسم الهيرو مستقل ومعقّد.

الأسطر 88–94

```
88    Widget build(BuildContext context) {
89      return Container(
90        clipBehavior: Clip.antiAlias,
91        decoration: const BoxDecoration(
92          color: AppColors.ink,
93          borderRadius: BorderRadius.vertical(bottom: Radius.circular(36)),
94        ),
```

ماذا يفعل؟ `Container` بلون `ink` وزوايا سفلية مدوّرة 36 مع `clipBehavior` .

لماذا بهذه الطريقة؟ `Clip.antiAlias` ضروري: شريط التطريز أعرض (408) من الشاشة (390)، فنقصّه عند الحواف بدل أن يتجاوزها. ونصف القطر 36 للأسفل فقط حسب Figma.

الأسطر 95–96

```
95        child: SafeArea(
96          bottom: false,
```

ماذا يفعل؟ `SafeArea` بلا حماية من الأسفل.

لماذا بهذه الطريقة؟ نحمي شريط الحالة والنتوءات من الأعلى فقط ليبدأ الشريط تحتها. السفل يهمّنا أن يبقى ملتصقًا بالمحتوى.

الأسطر 97–98

```
97          child: Padding(
98            padding: const EdgeInsets.only(bottom: 40),
```

ماذا يفعل؟ حشوة سفلية 40.

لماذا بهذه الطريقة؟ مسافة بين النص والحافة المدوّرة كما في التصميم.

الأسطر 99–100

```
99            child: Column(
100             children: [
```

ماذا يفعل؟ `Column` أبناؤه في الوسط.

لماذا بهذه الطريقة؟ الافتراضي هو التوسّط الأفقي، وهو المطلوب.

الأسطر 101–106

```
101               SvgPicture.asset(
102                 AppAssets.tatreezBand,
103                 width: double.infinity,
104                 height: 24,
105                 fit: BoxFit.cover,
106               ),
```

ماذا يفعل؟ شريط التطريز: عرض كامل، ارتفاع 24، `BoxFit.cover` .

لماذا بهذه الطريقة؟ الأصل بعرض 408 وارتفاع 24؛ `cover` يملأ عرض أي شاشة ويقصّ الزائد، فيطابق التصميم على 390 بالضبط ولا يترك فراغًا على الشاشات الأعرض.

الأسطر 107–108

```
107              Padding(
108                padding: const EdgeInsets.fromLTRB(24, 32, 24, 0),
```

ماذا يفعل؟ حشوة: 24 يمين/يسار و32 من الأعلى.

لماذا بهذه الطريقة؟ من `pt-32 px-24` (Figma).

الأسطر 109–111

```
109                child: Column(
110                  spacing: 14,
111                  children: [
```

ماذا يفعل؟ `Column` بمسافة 14 بين الأبناء.

لماذا بهذه الطريقة؟ مسافة التصميم بين الشخصية والشعار والنص.

الأسطر 112–116

```
112                    SvgPicture.asset(
113                      AppAssets.mascot150,
114                      width: 150,
115                      height: 150,
116                    ),
```

ماذا يفعل؟ الشخصية (Mascot) بحجم 150.

لماذا بهذه الطريقة؟ ملف SVG مُصدَّر بحجم 150 بالضبط.

الأسطر 117–120

```
117                    Row(
118                      mainAxisSize: MainAxisSize.min,
119                      spacing: 10,
120                      children: [
```

ماذا يفعل؟ `Row` بحجم أدنى ومسافة 10 للشعار والاسم.

لماذا بهذه الطريقة؟ يتوسّط الصف في العمود.

الأسطر 121–125

```
121                        SvgPicture.asset(
122                          AppAssets.logoOnDark,
123                          width: 40,
124                          height: 40,
125                        ),
```

ماذا يفعل؟ شعار «رِزمة» على الخلفية الداكنة بحجم 40.

لماذا بهذه الطريقة؟ هو أول عنصر فيظهر على اليمين من الكلمة (كما في Figma).

السطر 126

```
126                       Text('رِزمة', style: AppTextStyles.wordmark40),
```

ماذا يفعل؟ كلمة «رِزمة» بنمط `wordmark40`.

لماذا بهذه الطريقة؟ التشكيل (الكسرة) موجود في النص كما في التصميم.

الأسطر 127–128

```
127                     ],
128                   ),
```

ماذا يفعل؟ إغلاق الـ Row.

لماذا بهذه الطريقة؟ نهاية الصف.

الأسطر 129–133

```
129                   Text(
130                     'من المحتوى الذي يُقرأ… إلى التعلم الذي يُمارس.',
131                     textAlign: TextAlign.center,
132                     style: AppTextStyles.tagline18,
133                   ),
```

ماذا يفعل؟ الشعار اللفظي «من المحتوى الذي يُقرأ… إلى التعلم الذي يُمارس.» بنمط `tagline18` وبمحاذاة للمنتصف.

لماذا بهذه الطريقة؟ يلتفّ على سطرين كما في التصميم (ارتفاع السطر 1.6).

الأسطر 134–143

```
134                 ],
135               ),
136             ),
137           ],
138         ),
139       ),
140     ),
141   );
142   }
143 }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف `_Hero`.

الأسطر 145–151

```
145 class _ValuePoint extends StatelessWidget {
146   const _ValuePoint({
147     required this.asset,
148     required this.tileColor,
149     required this.title,
150     required this.subtitle,
151   });
```

ماذا يفعل؟ ودجت `_ValuePoint` يستقبل الأيقونة ولون الخلفية والعنوان والوصف.

لماذا بهذه الطريقة؟ فصلنا النقاط الثلاث في ودجت واحد لتجنّب تكرار نفس 15 سطرًا ثلاث مرات.

الأسطر 153–156

```
153   final String asset;
154   final Color tileColor;
155   final String title;
156   final String subtitle;
```

ماذا يفعل؟ الحقول الأربعة.

لماذا بهذه الطريقة؟ كلها `final`.

الأسطر 158–161

```
158   @override
159   Widget build(BuildContext context) {
160     return Row(
161       spacing: 14,
```

ماذا يفعل؟ `Row` بمسافة 14 بين المربع والنص.

لماذا بهذه الطريقة؟ ترتيب الصف: مربع الأيقونة أولًا (يمين) ثم النص.

الأسطر 162–163

```
162       children: [
163         IconTile(asset: asset, color: tileColor),
```

ماذا يفعل؟ يعرض `IconTile` بالحجم الافتراضي 48.

لماذا بهذه الطريقة؟ استعمال المكوّن المشترك.

الأسطر 164–172

```
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(title, style: AppTextStyles.body16Strong),
              Text(subtitle, style: AppTextStyles.caption13),
            ],
          ),
        ),
```

ماذا يفعل؟ `Expanded` يملأ باقي العرض، وفيه عمود: العنوان ثم الوصف، محاذاة `start`.

لماذا بهذه الطريقة؟ `start` في RTL = اليمين، فيبدأ النص من جهة الأيقونة. `Expanded` يمنع تجاوز النص الطويل عرض الشاشة (يلتفّ بدل أن يسبب overflow).

الأسطر 173–176

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 178–182

```
class _PageDots extends StatelessWidget {
  const _PageDots({required this.count, required this.current});

  final int count;
  final int current;
```

ماذا يفعل؟ ودجت `_PageDots` بحقلين: العدد `count` والحالي `current`.

لماذا بهذه الطريقة؟ ودجت عام يصلح لأي عدد صفحات.

الأسطر 184–189

```
  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      spacing: 6,
      children: [
```

ماذا يفعل؟ `Row` متمركز بمسافة 6 بين النقاط.

لماذا بهذه الطريقة؟ في RTL: أول نقطة على اليمين، وهي النقطة «الحالية» الأولى — يطابق Figma.

الأسطر 190–200

```
        for (var i = 0; i < count; i++)
          AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            width: i == current ? 22 : 8,
            height: 8,
            decoration: BoxDecoration(
              color: i == current ? AppColors.teal : AppColors.disabled,
              borderRadius: BorderRadius.circular(4),
            ),
          ),
      ],
```

ماذا يفعل؟ حلقة `for` تولّد النقاط: الحالية بعرض 22 وبلون teal، والبقية بعرض 8 ورمادي `disabled`، مع زوايا 4 وارتفاع 8.

لماذا بهذه الطريقة؟ `AnimatedContainer` يحرّك التغيّر بين النقطة والشريط عند تبدّل `current` لاحقًا (200 ملّي ثانية) دون كود إضافي. `for` داخل القائمة (collection for) تغني عن `map().toList()`.

الأسطر 201–203

```
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.